

1)

When the same key and F are used for both the encryption and MAC, an attacker can mount a chosen-ciphertext attack (CCA) on the scheme.

Suppose the attacker intercepts a ciphertext (c, t) and wants to forge a new ciphertext (c', t') that decrypts to a meaningful message. The attacker can perform the following steps:

Choose a random 2-block value IV and encrypt it using the given key to get $r = F(k, IV)$.

Modify the intercepted ciphertext by flipping some bits in the first block $c[1]$ to get $c'[1]$. Note that this modification does not affect the validity of t as a MAC on c .

Compute a new MAC t' as follows:

$$t' = \text{CBC-MAC}(k, c'[1] \parallel F(k, c'[1] \oplus r) \parallel c[2] \parallel t)$$

Send the forged ciphertext (c', t') to the decryption oracle. The oracle will decrypt it as follows:

$$r' = F(k, c'[1])$$

$$m = F(k, c'[1] \oplus r') \oplus c[2]$$

Output m as the forged message.

Since the attacker can modify the intercepted ciphertext arbitrarily and compute valid MACs on the modified ciphertexts using the same key and F , the scheme is not CCA-secure when the same key and F are used for both the encryption and MAC.

2)

The attacker has access to an oracle that computes CBC-MAC with the same key K for any message of any length. We will show how the attacker can forge a new message tag pair (m^*, t^*)

First, consider a message $m_1 = 0 \parallel m$. The attacker sends m_1 to the oracle and obtains the tag $t_1 = \text{CBC-MAC}(K, m_1)$. Since the first byte of m_1 is 0, the CBC-MAC computation initializes the IV to 0. Therefore, t_1 is the MAC of two blocks: the single block consisting of 0's, followed by the block consisting of m^* .

Next, consider another message $m_2 = m^* \parallel 0$. The attacker sends m_2 to the oracle and obtains the tag $t_2 = \text{CBC-MAC}(K, m_2)$. Since the last byte of m_2 is 0, the CBC-MAC computation will not use the last block of m^* . Instead, it will pad the last block of m^* with 0's and compute the MAC of two blocks: the block consisting of m^* , followed by a block of 0's.

Now, consider the message $m_3 = m^* \parallel x \parallel y$, where x and y are two arbitrary strings of bytes. The attacker sends m_3 to the oracle and obtains the tag $t_3 = \text{CBC-MAC}(K, m_3)$. The CBC-MAC computation processes m^* and the first half of x as the last block of the message, and then applies XOR to the IV and the last block to produce the tag.

Now, the attacker constructs a message m^* such that the last block of m^* is a concatenation of x and y , and the CBC-MAC tag t^* is a concatenation of the tags t_1 , t_2 , and t_3 . Since CBC-MAC is a deterministic function, the attacker can compute t^* without access to the oracle. The probability that m^* and t^* are accepted as a valid message tag pair by the receiver is non-negligible, because it depends only on the probability that the attacker can find two strings x and y that satisfy the condition $|x| = |y| > \lambda/2$. By the birthday paradox, this probability is roughly $1/2$ when λ is sufficiently large.

Therefore, we have shown that CBC-MAC is insecure for message space $M = (\{0,1\}^\lambda)^*$, because an attacker can forge a new message tag pair